Operating Systems Lab 8

Process calls and pipes inside xv6

Study brief for comprehension and oral defense. This supplements the formal UTM report and does not replace its required answers or evidence.

What the lab asks

Read the same process ideas inside a small teaching kernel. The required example forks a child, replaces the child's program with echo, then waits for termination. The pipe program sends one byte from parent to child and a reply back. The goal is to connect user-visible calls to the kernel code that implements their rules.

Plain terms

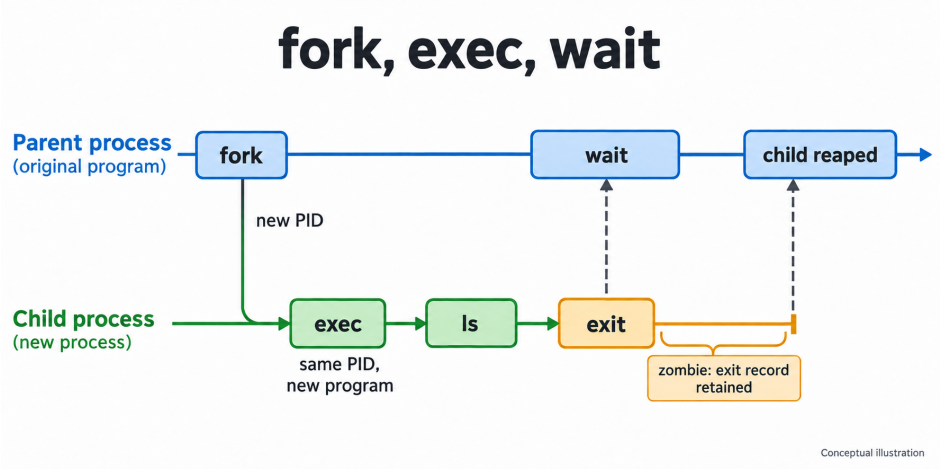
Kernel: privileged part of the OS that manages processes, memory and devices. **System call:** controlled request from user code to kernel code. **Address space:** memory view assigned to one process. **PID:** process identity. **Pipe:** kernel byte queue with one read end and one write end. **Blocking:** waiting for a required event such as data or child exit.

Read in ten minutes

2 min: draw parent and child after fork. 2 min: trace run.c through exec and wait. 2 min: follow user stub -> ecall -> dispatcher. 2 min: draw both pipe directions and close unused ends. 2 min: distinguish a pipe read from wait and test a changed condition.

Process lifetime

exec replaces a process image; wait later collects the child's exit status. The PID belongs to the process, so it survives a successful exec.



Trace one execution

Worked trace: user/run.c

The parent prints its PID and calls fork. The child receives 0, builds a null-terminated argv vector, and calls exec("echo", args). On success the same child process runs echo; on failure the call returns, the error line executes, and the child explicitly exits 1. The parent calls wait and prints the child's PID and status. The child's text reaches the console through inherited descriptors; wait returns process information, not the text.

Follow the system-call boundary

User code uses declarations in `user/user.h`. The `user/usys.pl` generator emits a stub that loads a syscall number in a7 and executes ecall. `kernel/trap.c` recognizes the user trap and calls `kernel/syscall.c`, whose table selects the system-call implementation. For process behavior, continue into `kernel/sysproc.c` and `kernel/proc.c`; for exec and pipes, follow `kernel/sysfile.c` to `kernel/exec.c` or `kernel/pipe.c`.

Captured behavior

The 19 Oct QEMU session showed parent PID 3, child PID 4, echo text and status 0. The missing-program variant reported status 1. These IDs and outputs describe one boot and one build. Report Part 1 is pages 5-6; the observations are pages 12-13.

Report and source anchors

Part 1: process and syscall concepts, report pages 5-6. **Variants:** pages 8-9. **QEMU observations:** pages 12-13. **Sources:** user/run.c and the xv6 user-to-kernel path.

Follow the connections

Worked connection: two pipes

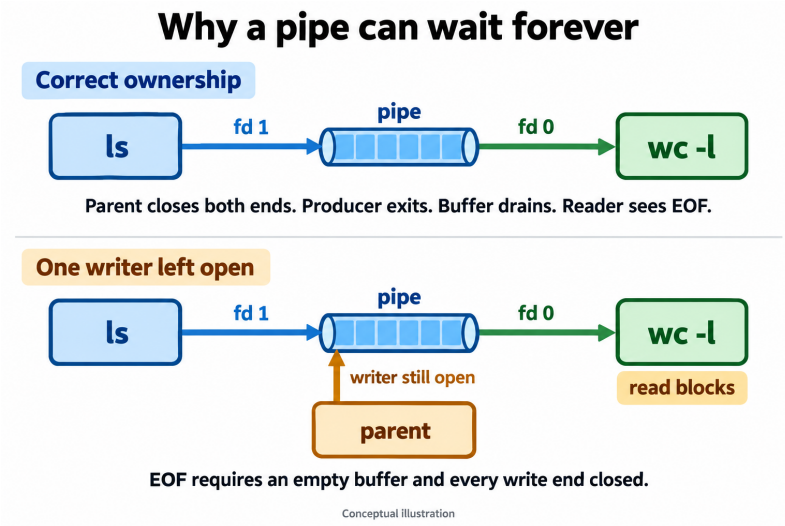
A pipe is one-way: fd[0] reads, fd[1] writes. `pingpong.c` creates parent-to-child and child-to-parent pipes before fork. Both processes inherit both directions, then close the endpoints they do not need. Parent writes one byte; child reads and prints ping, writes a reply; parent reads and prints pong. The parent's read synchronizes on data; its later wait synchronizes on child exit. The captured run printed child PID 10 receiving ping before parent PID 9 receiving pong. The numbers vary. The optional prime chain sent integers 2 through 35 and printed primes 2 through 31. It did not test 37 or values above 35. The source for read/write/close rules is `../lab2/kernel/pipe.c`; report Part 2 is pages 10-13 and optional primes is page 14.

Evidence limits

This is a recorded QEMU build and run, not a claim about every schedule or every pipe size. The generic pipe diagram illustrates the close/EOF rule for one direction; pingpong uses two separate pipes so each direction has a clear reader and writer.

Changed condition

If an unused writer copy stays open, an empty reader cannot observe EOF. If both directions shared one pipe, the parent could consume its own byte. A larger reply can also make wait-before-read deadlock when the pipe buffer fills.



This is a generic one-pipe ownership illustration. Pingpong uses two pipes, one for each direction; the close/EOF rule applies separately to each pipe.

Oral practice prompts

Answer these aloud before turning to the answer pages. Questions 1-15 cover the main required mechanisms; 16-20 test changed conditions and the limits of the evidence.

1. [Likely] In xv6 run.c, what does fork return in the child and the parent?

2. [Likely] After the child successfully execs echo, is it a new process with a new PID?

3. [Likely] Why does the line after exec print an error only when exec fails?

4. [Likely] What is argv in the run example, and why must its last pointer be null?

5. [Likely] Which process prints the echo words, and what does the parent get from wait?

6. [Likely] Why is failed exec status 1 in xv6 while the Linux Lab 7 example uses 127?

7. [Likely] What does wait do for the parent in this xv6 tree?

8. [Likely] Trace one xv6 system call from a user program toward the kernel dispatcher.

9. [Likely] The recording shows parent PID 3 and child PID 4. Should the next run show the same numbers?

10. [Likely] Which end of an xv6 pipe is read, and which is written?

11. [Likely] Why does pingpong create two pipes instead of using one for both directions?

12. [Likely] Trace the one-byte ping-pong order. What happens before the parent calls wait?

13. [Likely] Name three different calls that may block in this exercise and what each waits for.

14. [Likely] Why must each pingpong process close pipe descriptors it will not use?

15. [Likely] Why should the source check fork, pipe, read and write return values?

16. [Harder] In xv6 pipe.c, when does a read return zero rather than wait?

17. [Harder] The optional primes program feeds 2 through 35. Why does its output end at 31, and what would 2 through 50 add?

18. [Harder] If both pingpong directions shared one pipe, what race appears?

19. [Harder] Could the parent call wait immediately after sending ping and then read pong safely?

20. [Harder] Suppose the receive pipe is empty after the child writes, but another process retains its writer. What does the parent read observe?

Checking answers: questions 1-10

1. The child gets 0. The parent gets the child's positive PID. A negative return indicates failure to create the child.

Why: The same fork call creates two continuations whose return values select their branches.
Change: On failure, the parent must handle the negative value instead of waiting for a child that does not exist.
Trace: user/run.c:12-18; report Part 1, pp. 5-6.

2. No. exec replaces the program image inside the existing child process, so the PID remains the same.

Why: fork creates a process; exec changes what that process runs.
Change: If the child forked again instead of execing, there would be another process and another PID.
Trace: user/run.c:18-24; report Part 1, pp. 5-6.

3. A successful exec does not return to the old image. A failed exec returns, so the child prints the error and explicitly exits with status 1.

Why: The failure path follows the call because normal success replaces that path entirely.
Change: Changing the target to a valid program bypasses the error and exit(1) lines.
Trace: user/run.c:18-24; report Part 1, pp. 5-6; variants, pp. 8-9.

4. argv is an array of pointers to argument strings. xv6 exec expects a null-terminated vector so it can identify where the arguments end.

Why: The program receives arguments as a sequence rather than as one shell command string.
Change: Changing one string changes what echo prints; omitting the null terminator makes the vector invalid.
Trace: user/run.c:19-20; user/echo.c:6-18; report Part 1, pp. 5-6.

5. The child, now running echo, prints the words. The parent gets the terminated child's PID and exit status from wait; it does not receive the child's text through wait.

Why: Output uses inherited console descriptors, while wait communicates process termination information.
Change: Redirecting the child's stdout would change where the words appear without changing the parent's wait result.
Trace: user/run.c:18-32; user/echo.c:6-18; report Part 1, pp. 5-6.

6. Each child chooses an explicit exit value after exec returns with failure. Lab 8 calls exit(1); Lab 7 calls exit(127). exec does not choose either value.

Why: The status is application policy layered on the failed system call.
Change: Changing the explicit exit argument changes the value wait later reports.
Trace: user/run.c:22-24; ../lab7/run.c:18-20; report Part 1, pp. 5-6; variants, pp. 8-9.

7. It waits for a child to exit, returns that child's PID and copies its exit status to the supplied address; it also releases the zombie process slot.

Why: The kernel separates child exit from the parent's collection of its status.
Change: If the child has not exited yet, wait blocks; if the parent has no waitable child, the return path differs.
Trace: user/run.c:27-32; ../lab2/kernel/proc.c:368-397; report Part 1, pp. 5-6.

8. The program uses a declaration in user.h; usys.pl generates a stub that puts the syscall number in a7 and executes ecall; usertrap enters the syscall dispatcher, which selects a sys_* handler.

Why: The wrapper gives ordinary user code a controlled path across the user/kernel privilege boundary.
Change: A different call uses a different syscall number and dispatcher entry, but the boundary pattern remains.
Trace: ../lab2/user/user.h:6-14; usys.pl:9-21; kernel/trap.c:38-68; kernel/syscall.c:132-142; report Part 1, pp. 5-6.

9. No. Those are measured values from one QEMU boot. Process IDs depend on the current xv6 process table and previous commands.

Why: PID order is runtime state, while parent-child roles follow the fork return values.
Change: Running another command first may consume IDs and shift both values.
Trace: user/run.c:10-21; report QEMU observations, pp. 12-13; execution session, 19 Oct 2026.

10. fd[0] is the read end and fd[1] is the write end. One process can write bytes and another can read them in order.

Why: The two descriptors represent different sides of one kernel-managed byte stream.
Change: Swapping the roles without changing the calls makes a process read from the wrong endpoint or write to the wrong one.
Trace: user/pingpong.c:7-8; ../lab2/kernel/sysfile.c:478-497; report Part 2, pp. 10-11.

Checking answers: questions 11-20

11. One pipe carries parent-to-child ping and the other carries child-to-parent pong. Separate streams avoid a process consuming its own byte before the peer does.

Why: A pipe is one-way; two independent directions need two independent channels for this protocol.
Change: A more complex single-pipe protocol would need extra coordination or message framing to prevent self-read and ordering races.
Trace: user/pingpong.c:7-24; report Part 2, pp. 10-11.

12. The parent writes the ping byte and closes that writer. The child reads it, prints ping, writes the reply and closes its writer. The parent reads and prints pong, then waits for the child's exit.

Why: The pipe read is the synchronization for the reply; wait is process-lifetime synchronization afterward.
Change: If the parent reads before sending the first byte, the child and parent can both wait for data and deadlock.
Trace: user/pingpong.c:31-74; report Part 2, pp. 10-13.

13. The child's first read waits for ping; the parent's second read waits for pong; the parent's wait waits for child termination. These are distinct conditions.

Why: Blocking describes waiting for an event, but the event can be pipe data or process exit.
Change: A read can return immediately if data is buffered; wait can return only after a child is waitable.
Trace: user/pingpong.c:34,62,70-74; report Part 2, pp. 10-11.

14. Unused copies waste descriptors and can keep a writer open. A reader gets EOF only after buffered bytes drain and all write references are closed.

Why: fork duplicates descriptor references, so both processes initially hold both directions.
Change: Leaving one inherited writer open can make a reader wait indefinitely after the intended writer exits.
Trace: user/pingpong.c:31-68; ../lab2/kernel/pipe.c:59-74,106-129; report Part 2, pp. 10-11.

15. These calls can fail or transfer fewer bytes than requested. Checking the result prevents the program from treating a failed setup or incomplete transfer as success.

Why: Operating-system calls report both success and failure through their return values.
Change: If a read returns 0 at EOF, retrying forever is also wrong; the program must interpret zero as no more data.
Trace: user/pingpong.c:7-29,34-74; report Part 2, pp. 10-11.

16. When the pipe has no buffered bytes and no write endpoint remains open. While a writer remains, an empty read waits for data.

Why: EOF means there can be no future bytes, not merely that the buffer is momentarily empty.
Change: If one inherited writer stays open, the empty reader can continue waiting instead of returning zero.
Trace: ../lab2/kernel/pipe.c:106-129; report Part 2, pp. 10-11.

17. 31 is the largest prime in the tested input range. Extending the producer through 50 would also send 37, 41, 43 and 47 through the filters.

Why: The filter only reports values it receives; it cannot emit a prime that the producer never sends.
Change: Changing the upper bound changes the tested output, not the prime-filter rule.
Trace: user/primes.c:7-28,112-126; report Optional primes, p. 14; execution session, 19 Oct 2026.

18. After the parent writes ping, its own read and the child's read would compete for the same bytes. The parent could consume its own ping, leaving the child blocked.

Why: A pipe delivers bytes to whichever reader consumes them first; it has no sender/receiver identity.
Change: Separate pipes give each direction one clear reader and writer.
Trace: user/pingpong.c:7-24; report Part 2, pp. 10-11.

19. For this one-byte reply, likely yes because it fits in the pipe buffer and the child can exit. For a larger reply, the child could fill the pipe and block writing while the parent blocks in wait, so neither progresses.

Why: wait synchronizes on process exit, while reading frees pipe-buffer space; ordering can create a deadlock.
Change: If the parent reads concurrently or before waiting, a large response can drain and let the writer finish.
Trace: user/pingpong.c:55-74; ../lab2/kernel/pipe.c:77-102; report Part 2, pp. 10-11.

20. It cannot see EOF yet. Once buffered data is consumed, the empty read waits because the retained writer means more data may still arrive.

Why: The kernel counts open writer references, not the number of processes that intend to write.
Change: Closing the final writer lets the read return zero after existing bytes drain.
Trace: ../lab2/kernel/pipe.c:59-74,106-129; report Part 2, pp. 10-11.